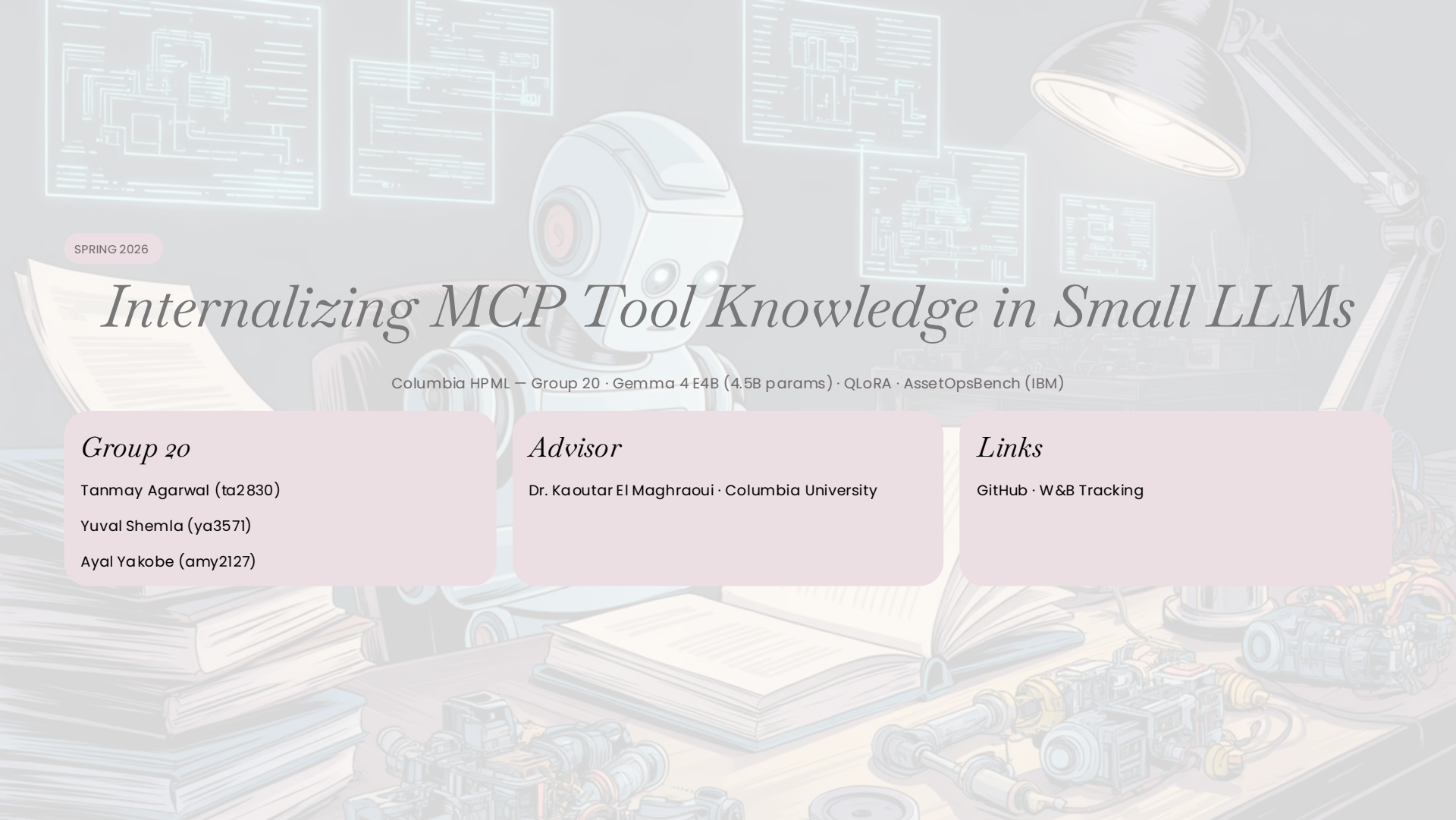




SPRING 2026

Internalizing MCP Tool Knowledge in Small LLMs

Columbia HPML — Group 20 · Gemma 4 E4B (4.5B params) · QLoRA · AssetOpsBench (IBM)

Group 20

Tanmay Agarwal (ta2830)

Yuval Shemla (ya3571)

Ayal Yakobe (amy2127)

Advisor

Dr. Kaoutar El Maghraoui · Columbia University

Links

GitHub · W&B Tracking

Motivation & Problem

Why this matters

AssetOpsBench evaluates whether an LLM can answer industrial operations questions by choosing the right MCP agents, tools, arguments, and step order.

In the standard setup, every planning prompt includes the full tool catalog, increasing latency, cost, and context usage.

Each tool requires ~100–200 tokens of description — with 20+ tools, this adds up fast.

Can we teach a model to remember the tools?

The Token Problem

~100–200 tokens

Per tool

Fixed cost per tool description in prompt

573 tokens

Informed prompt

Full catalog included every query

76% saved

Our approach

Scales to 5,000–10,000+ tokens with larger catalogs

Our Approach — Fine-Tuning a Small Planner

Plan format: #Task → #Agent → #Tool → #Args → #Dependency per step

Why Fine-Tune?



Benefits

- Internalize tools into weights
- Eliminate prompt overhead
- Faster inference



Risks

- Catastrophic forgetting
- Overfitting
- Data contamination

All three risks addressed with clean splits, early stopping, and forgetting benchmarks.

What the model must learn

1

Agent Awareness

Learn which agents exist and their roles

2

Tool Ownership

Map tools to the correct agent

3

Argument Schema

Know what each tool expects

4

Multi-Step Chaining

Compose plans with correct dependencies

Technical Setup

Model: Gemma 4 E4B

- ~4.5B parameters, instruction-tuned
- PyTorch + Transformers + PEFT + TRL
- Fine-tunable on a single A100 GPU
- Already understands structured output formats

QLoRA Fine-Tuning

- Only 0.5-5% trainable parameters
- LoRA Rank 16 (optimal from sweep $r=8/16/32/64$)
- All linear layers targeted
- Alpha 32, dropout 0.05
- AdamW + cosine LR, early stopping patience=2

Hardware & Data

- 1x NVIDIA A100 40GB
- 8-bit model loading (~8GB), 4-bit (~5GB)
- Batch size 1, grad accumulation 8, max seq 512
- 122 train / 30 test scenarios
- W&B logging + Colab execution

Optimizations & Evaluation Methodology

Optimizations Applied

1

Pattern-aware clean split — Rebuilt split at scenario level; eliminated 100% train/test leakage.

2

QLoRA fine-tuning — Only 0.63% of params trained on a single GPU.

3

Structured evaluation — Multi-dimensional metrics capturing both exactness and quality.

What We Track

Profiling Tools

- Weights & Biases — training runs, loss curves, hyperparameters
- Notebook instrumentation — per-section timing
- Custom MCP parser — plan format validation
- Gemini 2.5 Flash — qualitative judge scoring

Metrics

- Training memory: peak GPU usage (8-bit vs 4-bit)
- Training time: wall-clock per epoch
- Inference throughput: latency per call (30-call benchmark)
- AT-F1: agent+tool selection accuracy
- ArgKey-F1: argument field accuracy
- Judge score: 6-dimension LLM evaluation (1-5)

Data Pipeline & Training

Dataset Composition



Tool Knowledge

Agent/tool ownership, arguments, hard negatives

Planning Pairs

Multi-step plans from Gemini 2.5 Flash



Execution Traces

Full end-to-end task traces

Train / Test Split

Scenario-level, leakage-free

Training: Config C (Best)

1,741 examples

Tool Knowledge + Planning combined

~435 steps, 2 epochs

Tool structure internalized in first ~300 steps

Early stopping

Catches optimal point before validation loss climbs

Best checkpoint

Step ~435

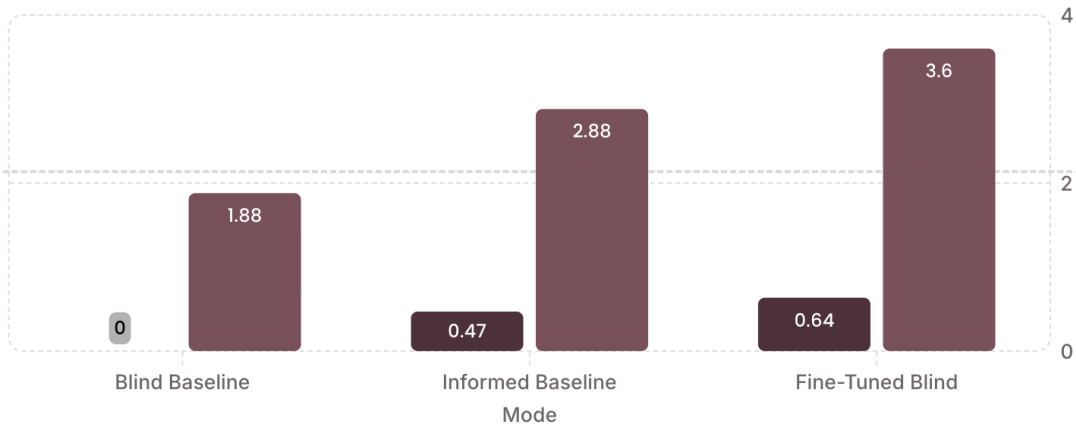


🏆 Main Results

Results — The Model Can Learn Tools

30-Scenario Held-Out Evaluation

■ AT-F1 ■ Judge Score (/5)



76%

Fewer tokens

573 → 139

+35%

AT-F1

0.470 → 0.635

+25%

Judge Score

2.88 → 3.60

Agent Routing

0% → 97.5%

Tool Selection

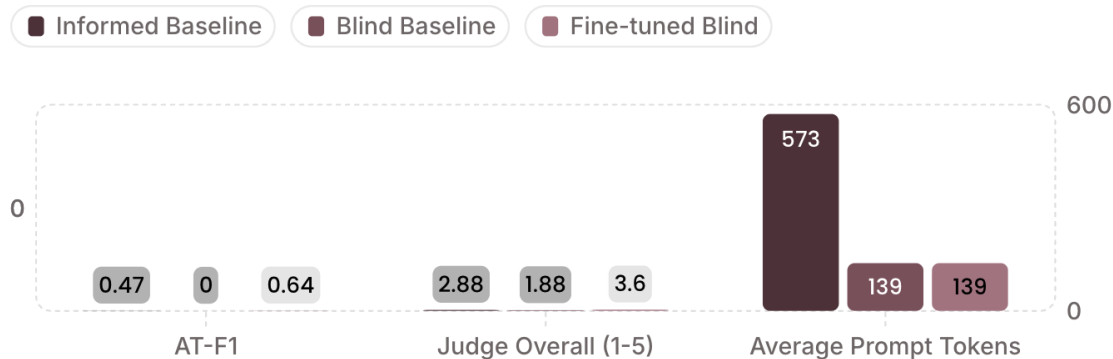
0% → 94.9%

Token Savings

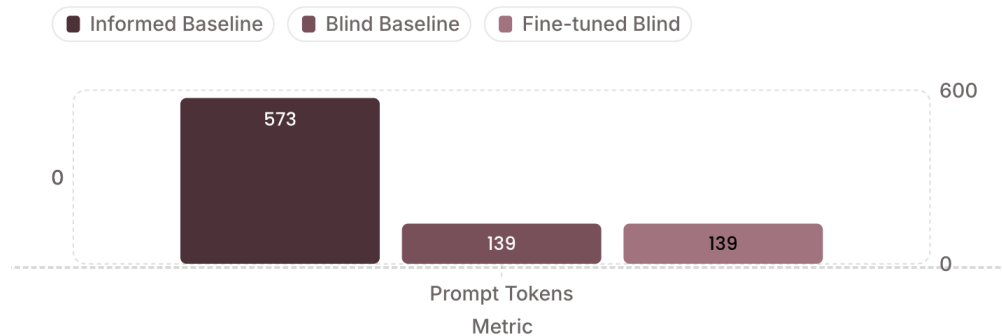
76% fewer

Results — Before vs. After

Quality Metrics



Prompt Tokens



Takeaway

Base model collapses

Blind baseline AT-F1 = 0.000 without fine-tuning

Fine-tuning recovers

Tool+Plan training surpasses the informed baseline

Quality + efficiency

Higher planning quality with 76% fewer prompt tokens

Ablation Studies

Key Findings

Data Ablation

Config C wins (Tool Knowledge + Planning). Neither alone is sufficient – the model needs both tool awareness and plan composition.

LoRA Rank Sweep

Tested $r=8, 16, 32, 64$. **Diminishing returns after $r=16$.**

Quantization

8-bit = better training stability. 4-bit = 29% faster training, 34% faster inference.



LoRA Rank Ablation

Trainable Parameters

$r=8$

25.2M (0.32% trainable)

$r=16$

50.5M (0.63% trainable)

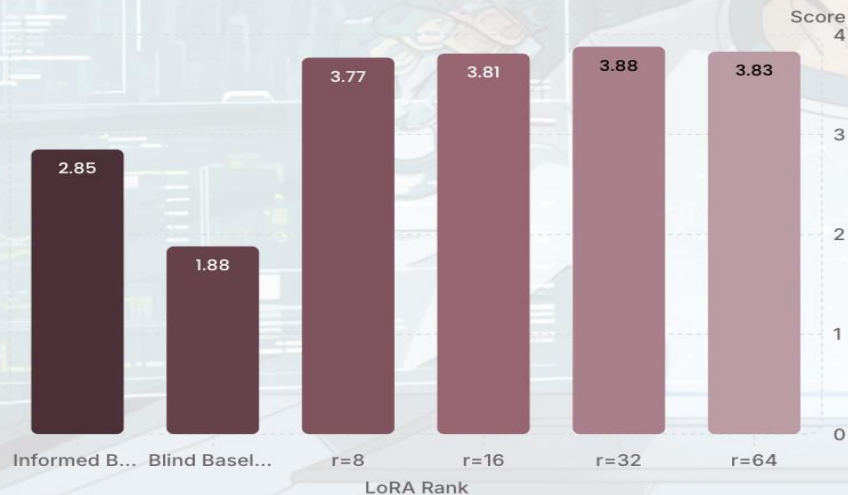
$r=32$ (BEST)

101.0M (1.26% trainable)

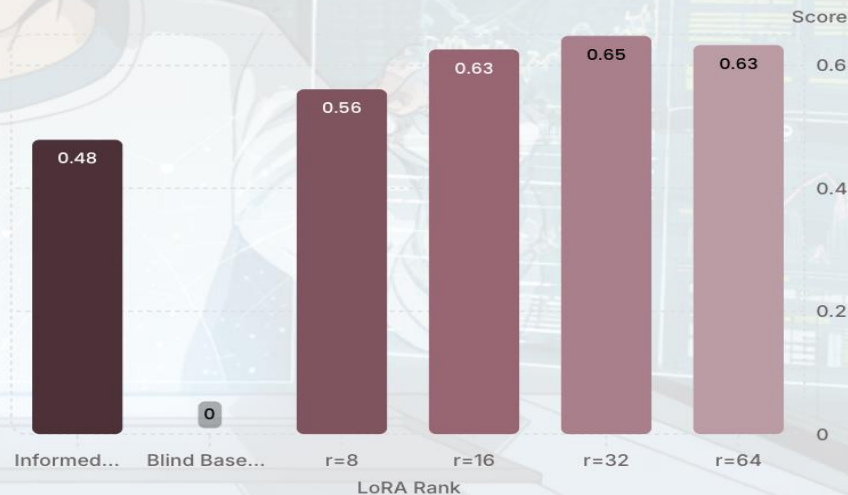
$r=64$

202.0M (2.48% trainable)

Overall Judge Score (1-5)



AT-F1 Score (0-1)

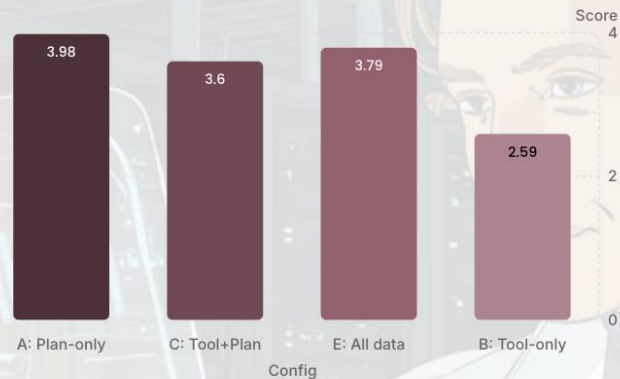


Key takeaway: $r=32$ is the sweet spot — best judge score and AT-F1. Diminishing returns at $r=64$.

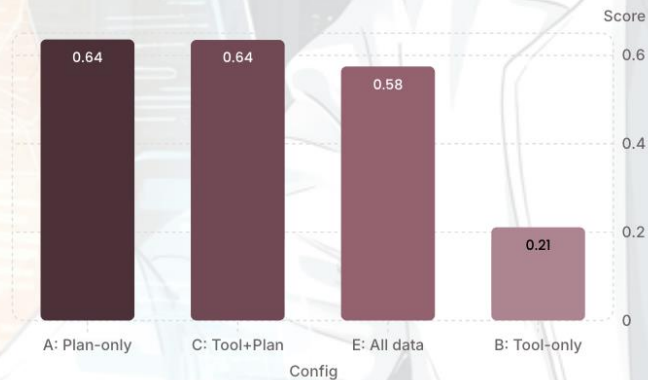
Data Ablation

Which training data matters most? Tool knowledge, planning examples, or both?

Overall Judge Score (1-5)



AT-F1 Score (0-1)



Full Data Ablation Comparison

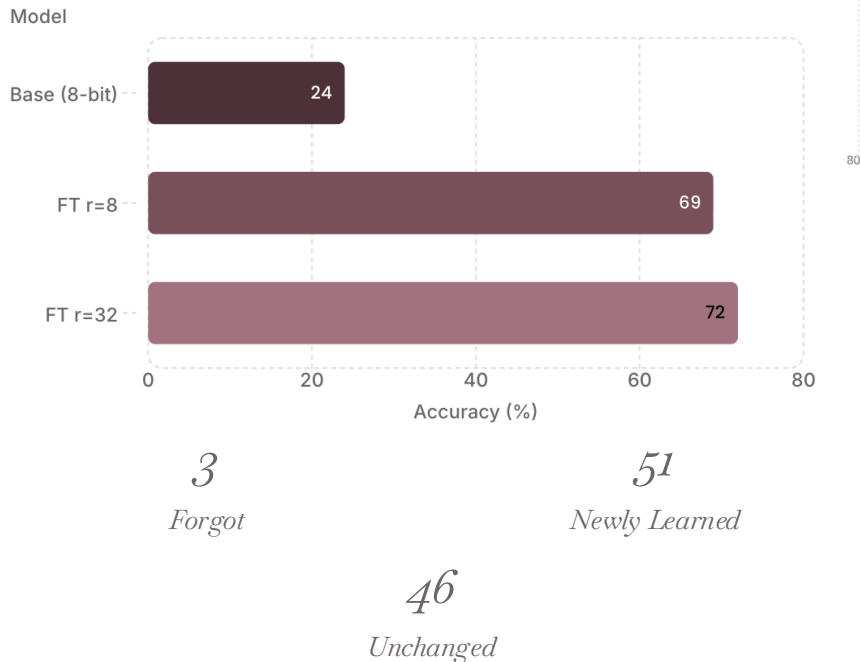
A: Plan-only	1584	0.636	3.98	2.83	3.87
3.97	3.45	Best judge — plans contain implicit tool knowledge			
C: Tool+Plan	1741	0.635	3.60	2.83	4.31
3.97	3.45	Best routing & tool selection			
data	1816	0.575	3.79	2.67	4.48
3.73	Best routing but execution traces dilute plan quality				
B: Tool-only	307	0.211	2.59	1.63	3.13
1.80	2.27	Knows tools exist but can't compose plans			

Catastrophic Forgetting Analysis

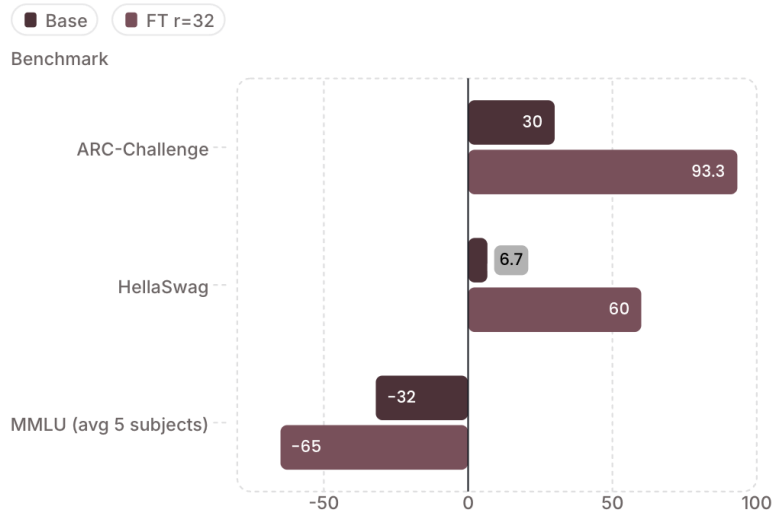
100 MCQ from MMLU (40), ARC-Challenge (30), HellaSwag (30) — base model vs fine-tuned on MCP data

Important caveat: The base model scores 24% (near-random) while fine-tuned models score 69–72%. This shows improvement, not forgetting. The likely cause: 8-bit quantization degrades the base model, and LoRA adapters partially repair it during training. A log-probability evaluation is pending to verify.

Overall MCQ Accuracy



Per-Benchmark Accuracy (r=32 vs Base)



Key insight: Fine-tuning on tool planning does not cause catastrophic forgetting — it actually improves benchmark performance, likely due to LoRA repairing quantization artifacts.

KEY TAKEAWAYS

Key Takeaways & Lessons Learned

What We Proved

- Small models CAN internalize tools — 4.5B params + QLoRA works.
- 76% fewer tokens, better quality — No tool descriptions needed at inference.
- Converges in ~300 steps — Early stopping prevents overfitting.
- Fine-tuned blind beats informed baseline — AT-FI: **0.635 vs 0.470**
- Combined data is essential — Tool knowledge + planning, neither alone suffices.



What worked

- Clean train/test separation mattered most
- Tool+Plan data combination beat either source alone
- Structured output format enables strong planning in small models
- Data quality > data quantity — rebuilding the split was the key unlock

Limitations and Future Work



Current Limitations

- Single domain (AssetOpsBench only)
- Implemented a subset of AssetOpsBench agents
- Tested Gemma 4 E4B only
- 30 test scenarios — limited statistical power



Future Directions

- Continual learning: add agents without forgetting
- Curriculum SFT: tool knowledge → planning → execution
- Specialist architecture: router + per-agent adapters
- Test on MCP-Universe for cross-domain generalization
- Deploy as planner module in production MCP pipelines

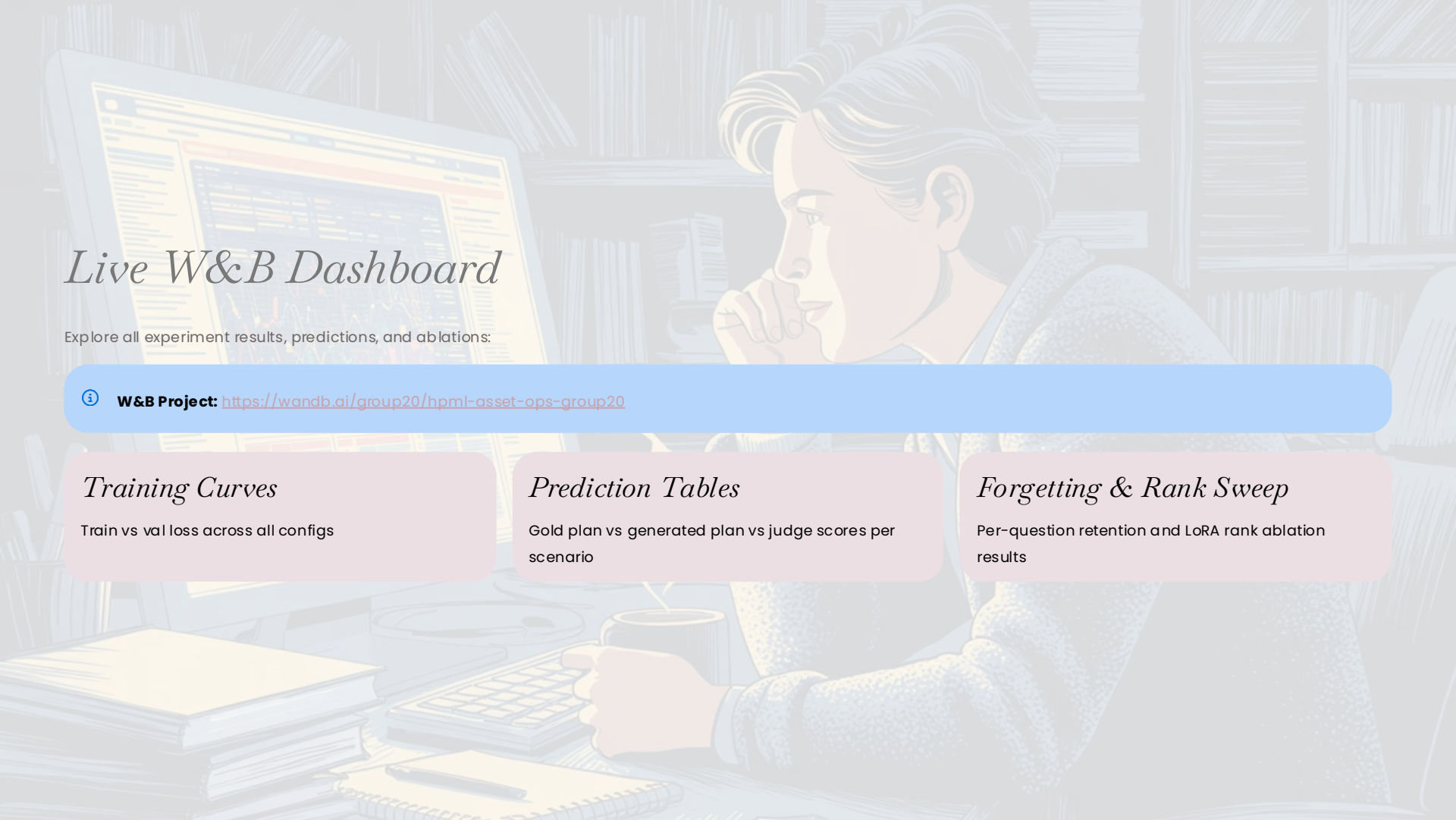
Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Tanmay Agarwal	Fine-tuning experiments, Colab execution, presentation	Planner notebooks, debugging, final deck
Yuval Shemla	Benchmark setup, data pipeline, AssetOpsBench integration	Repo, datasets, benchmark setup
Ayal Yakobe	Evaluation, W&B tracking, analysis	W&B dashboard, judge analysis, result summaries
Team	Paper framing, result reviews, rehearsal	HPML report, talk flow, submission

Thank you — Questions?

[GitHub](https://github.com/YuvalShemla/hpml-2026-project): <https://github.com/YuvalShemla/hpml-2026-project>

[W&B](https://wandb.ai/group20/hpml-asset-ops-group20): <https://wandb.ai/group20/hpml-asset-ops-group20>



Live W&B Dashboard

Explore all experiment results, predictions, and ablations:



W&B Project: <https://wandb.ai/group20/hpml-asset-ops-group20>

Training Curves

Train vs val loss across all configs

Prediction Tables

Gold plan vs generated plan vs judge scores per scenario

Forgetting & Rank Sweep

Per-question retention and LoRA rank ablation results

Backup — Experimental Setup

Setting	Baseline	Optimized
Batch size	Eval only	Batch 1, grad accum 8
Precision	8-bit inference	8-bit + LoRA adapters
Learning rate	n/a	2e-4, cosine, warmup 0.05
Sequence length	512	512
Epochs / steps	n/a	3 epochs, 654 steps
Hardware	1× A100 40GB	1× A100 40GB
Software	PyTorch + Transformers	PyTorch + PEFT + TRL + bitsandbytes + W&B

Evaluation Metrics Reference

Metric	What it measures	Scale
AT-F1	Agent-Tool pair F1 — correct agent+tool selected?	0–1
ArgKey-F1	Argument key F1 — correct argument names used?	0–1
Judge Overall	Gemini 2.5 Flash average across 6 dimensions	1–5
Judge Correctness	Would this plan answer the question?	1–5
Judge Tool Selection	Are the correct tools selected?	1–5
Judge Agent Routing	Are the correct agents assigned?	1–5
Judge Arg Quality	Are tool arguments correct?	1–5
Judge Efficiency	Is the plan appropriately sized?	1–5
Judge Dependency	Are step dependencies correct?	1–5

Found this useful? Share with your team or tag someone working on LLM tool-use.